

并行程序设计 Lab4: AI 使用报告

刘蔚霖 2412727

2026 年 6 月 9 日

1 关键对话

用户: 阅读 Lab4 要求和口令猜测代码, 规划 MPI 加速方案。Backup 保持不动, 并分别实现基础版和进阶版。

AI: 基础版使用阻塞式 Master-Worker。Rank 0 维护唯一 PT 优先队列, 将完整 PT 动态分配给空闲 Worker; Worker 在本地生成候选口令并执行 MD5。进阶版使用生成线程、有界缓存和非阻塞 MPI, 将候选生成、传输与加密组织为流水线。

用户: Base 版本具体如何满足 Master-Worker 要求?

AI: Rank 0 作为 Master, 独占概率优先队列并调用 `PopNextTask()` 取得概率最高的预终结式 (Pre-Terminal, PT)。Master 将完整 PT 封装成 `WorkUnit` 后发送给 Worker。Worker 调用 `GenerateRange()` 在本地生成候选口令, 并执行 NEON MD5, 完成后只返回处理数量和计时统计。

用户: Base 为什么采用完整 PT 作为任务, 而不发送生成后的候选口令?

AI: PT 描述比完整候选字符串批次紧凑。发送 PT 后, Worker 可以利用本地训练好的 PCFG 模型独立派生候选口令, 节点间只传输任务描述和统计结果。该数据分配方式减少网络传输量, 并将候选生成与 MD5 计算同时分散到多个 Worker。

用户: Base 如何处理不同 PT 工作量差异造成的负载不均衡?

AI: Master 使用 `MPI_ANY_SOURCE` 接收任意 Worker 的完成结果。最先完成的 Worker 会立即获得下一个 PT, 因此执行较短任务的 Worker 可以连续领取工作。该策略属于动态任务调度, 能够缓解完整 PT 大小不同造成的静态分配不均衡。

用户: `WorkUnit` 为什么需要单独设计?

AI: `WorkUnit` 同时保存 PT 和最后一个 segment 的半开区间 `[begin, end)`。Base 使用覆盖完整 PT 的范围; Advanced 可以将大型 PT 切分为多个互不重叠的范围任务。统一的数据结构使两个版本共享范围生成接口, 并保证切分任务之间没有重复或遗漏。

用户：PT 和字符串含有 STL 动态数据，MPI 应该怎样发送？

AI：不能直接发送 C++ 对象的内存布局。使用 `MPI_Pack` 按固定顺序写入 segment 数量、类型、长度、索引和范围边界，接收端再以相同顺序调用 `MPI_Unpack`。这种字段级序列化传输逻辑内容，不依赖对象中的指针地址。

用户：Advanced 版本的实现思路是什么？

AI：Advanced 将流水线分为候选生成、MPI 调度和加密三个部分。Rank 0 启动多个 Pthread 生成线程，每个线程拥有独立任务队列和有界 READY 缓存。生成线程将 `WorkUnit` 展开成口令批次；Rank 0 主线程轮询 READY 缓存并发送批次；其他 Rank 作为加密 Worker，接收批次并执行 NEON MD5。

用户：为什么生成线程不能直接调用 MPI？

AI：程序通过 `MPI_Init_thread` 请求 `MPI_THREAD_FUNNELED`。该线程级别规定只有初始化 MPI 的主线程调用 MPI 接口。生成线程只负责候选生成和缓存操作，Rank 0 主线程统一处理发送、接收和请求完成，从而保持明确的 MPI 调用边界。

用户：Advanced 如何给多个生成线程分配任务？

AI：Rank 0 从优先队列连续取得 PT，并使用 `SplitTask()` 将大型 PT 切分成范围任务。程序根据 `GuessCount()` 估计每个 `WorkUnit` 的候选数量，将下一任务分配给预计累计负载最小的生成线程。该策略以候选数量近似生成工作量。

用户：Advanced 为什么使用非阻塞 MPI？

AI：Rank 0 需要同时维护多个加密 Worker，并让通信与候选生成、MD5 计算重叠。主线程使用 `MPI_Isend` 发送批次，使用 `MPI_Irecv` 接收统计结果，再通过 `MPI_Testany` 和 `MPI_Waitany` 处理最先完成的 Worker。每个 Worker 使用独立发送缓冲区，缓冲区在对应请求完成前保持有效。

用户：为什么 Advanced 设计更复杂，但服务器结果可能慢于 Base？

AI：Base 发送紧凑 PT 描述，Worker 在本地生成并哈希。Advanced 发送已经生成的完整候选字符串，每条口令都需要打包长度和字符内容。非阻塞通信能够重叠部分等待时间，但无法消除序列化和数据传输成本。当通信时间占墙钟时间的比例较高时，增加加密 Worker 只能缩短哈希阶段，整体性能会进入通信饱和区间。

用户：为什么增加生成线程不一定继续加速？

AI：增加生成线程只缩短候选生成阶段。当生成端已经能够持续提供 READY 批次后，墙钟时间主要由 Rank 0 通信和加密 Worker 决定。额外生成线程还会与 MPI 主线程争用处理器和内存带宽，因此生成关键路径可能继续下降，而整体墙钟时间保持不变或上升。

用户：批次越大是否一定越快？

AI：较大批次可以减少消息次数，但会延长批次发布前的组装等待，并增加尾部任务不均衡。较小批次能够更早发送，使生成、通信和加密保持细粒度重叠。最佳批次大小取决于消息启动开销、序列化成本和 Worker 处理时间之间的平衡。

用户：程序如何保证所有 Worker 正常退出？

AI：Master 在任务耗尽后向每个 Worker 发送独立停止消息。Worker 根据消息标签区分工作任务与停止任务，退出接收循环后进入 `MPI_Finalize`。Advanced 主线程还会等待所有未完成发送和结果接收请求结束，再发送停止消息并释放线程和缓存。

用户：Base 输出显示处理数量超过 `--generate`，是否有问题？

AI：程序按完整 PT 或完整范围任务处理，并在累计数量越过目标后停止，因此最终处理数量可能略高于目标值。不同进程数和参数配置得到一致的正式处理数量，说明动态调度、任务切分和 MPI 消息没有造成任务重复或遗漏。